Design Journal Kaleidoscope

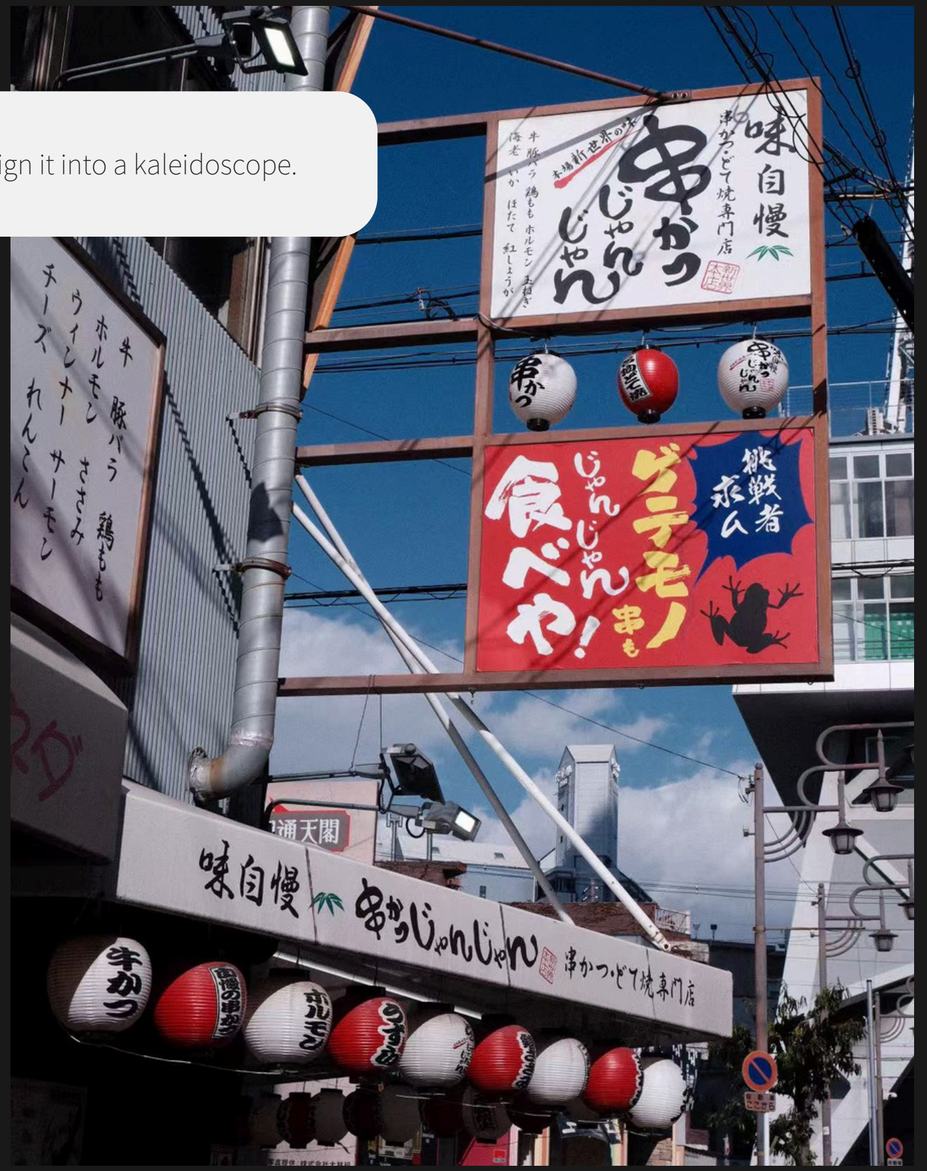
Homework3

CS208

Yihong Lin

Concept

Upload an image and redesign it into a kaleidoscope.



I loved to play kaleidoscope in my childhood , so when I heard our task was to transform the photo, the first idea came to my mind was to transform the photo into a kind of kaleidoscope view with color pixels copied and pasted. Since kaleidoscope is relied on the specular reflection, I planned to use symmetrical color block to realize it. In addition, by clicking different parts of the mouse and keyboard, people can have various views and perspectives of the photo.

Generative rules

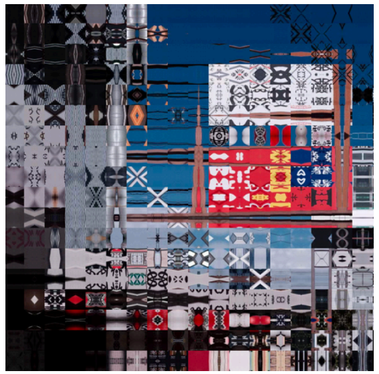
01

Original graphic (can be changed)

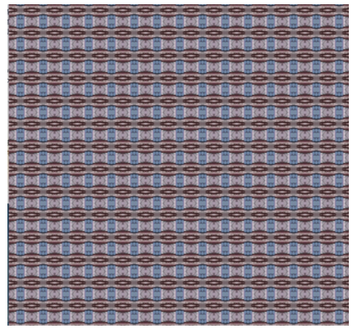


02

Transformation objective



Split image and symmetrical processing, can change the size of tiles



Kaleidoscope: extract and copy specific tile to fill the canvas

03

Transformation techniques

- Use the **get()** method to extract a sub-image of the specified position and size from the original image. Through nested **`for` loops** and the **`image()`** method, copy and fill the extracted sub-image onto the entire canvas.
- In the function **`makeTile()`**, use the **`get()`** method to obtain the pixel color , and use the **`set()`** method to copy the pixel color to four symmetric positions of the image to achieve symmetric transformation of the image.
- The value of tileSize can be dynamically adjusted by pressing the up or down arrow keys. After each adjustment of tileSize, the code will re-divide the image into small tiles of size tileSize x tileSize and perform symmetry processing on each tile (by calling the makeTile() function).

04

Anticipated outcome

- **Restore original image:** When the middle mouse button is pressed, the canvas will be restored to its initial state.
- **Copy specific tile to fill the canvas:** When the left mouse button is pressed, a 48x48 pixel tile at the current mouse position is extracted, symmetrical processing is performed on it, with the image changing according to the movement position.
- **Split image and symmetrical processing:** When the right mouse button is pressed, the entire image is split into multiple small tiles of size tileSize x tileSize, symmetrical processing is performed on each small tile.
- **Dynamically adjust image segmentation granularity:** By pressing the up or down arrow keys, the size of tileSize can be dynamically adjusted to change the granularity of image segmentation.

Code

```
int tileSize=20;
color pixelColor;
PImage Image;

void setup() {
  size(800, 800);
  Image=loadImage("image.jpg");
  Image.resize(width, height);
  image(Image, 0, 0);
}

void draw() {
  if (mousePressed) {
    if (mouseButton == CENTER) {
      image(Image, 0, 0, width, height);
    }
    if (mouseButton == LEFT) {
      for (int i = 0; i < width; i += 48) {
        for (int j = 0; j < height; j += 48) {
          image(makeTile(Image.get(mouseX, mouseY, 48, 48)), i, j);
        }
      }
    }
    if (mouseButton == RIGHT) {
      for (int i = 0; i < width; i += tileSize) {
        for (int j = 0; j < height; j += tileSize) {
          image(makeTile(Image.get(i, j, tileSize, tileSize)), i, j);
        }
      }
    }
  }
}

PImage makeTile(PImage outputImage) {
  for (int i=0; i<=outputImage.width/2; i++) {
    for (int j=0; j<=outputImage.height/2; j++) {
      pixelColor = outputImage.get(i, j);
      outputImage.set(i, j, pixelColor);
      outputImage.set(outputImage.width-i, j, pixelColor);
      outputImage.set(i, outputImage.height-j, pixelColor);
      outputImage.set(outputImage.width-i, outputImage.height-j, pixelColor);
    }
  }
  return outputImage;
}
```

```
void keyPressed() {
  if (key == CODED) {
    if (keyCode == UP) {
      tileSize += 8;
      for (int i=0; i<width; i+=tileSize) {
        for (int j=0; j<height; j+=tileSize) {
          image(makeTile(Image.get(i, j, tileSize, tileSize)), i, j);
        }
      }
    }
    if (keyCode == DOWN) {
      if (tileSize - 8 > 8) {
      tileSize -= 8;

      for (int i = 0; i < width; i += tileSize) {
          for (int j = 0; j < height; j += tileSize) {
              image(makeTile(Image.get(i, j, tileSize, tileSize)), i, j);
          }
        }
      }
    }
  }
}
```

if (mousePressed) : checks whether the mouse is pressed.
if (mouseButton == CENTER) : If the middle mouse button is pressed, redraw the original image onto the canvas.
if (mouseButton == LEFT) : If the left mouse button is pressed, copy the 48x48 pixel block at the current position of the mouse and fill the entire canvas. The makeTile() function is called here to process the block.
if (mouseButton == RIGHT) : If the right mouse button is pressed, the image is divided into several small blocks in tileSize, and the makeTile() function is called for each small block to process, and then the processed block is drawn on the canvas.

This function accepts an argument of type PImage, outputImage, which represents the block to be processed.
Traverse the upper left quarter of the block with two layers of nested for loops.
pixelColor = outputImage.get(i, j) : Gets the color of the current pixel.
outputImage.set() : Copies the color of the current pixel to the four symmetric positions of the block, thus achieving a symmetrical effect.
Finally, the processed block is returned.

if (key == CODED) : check whether a special key (such as arrow keys) is pressed.
if (keyCode == UP) : If the up arrow key is pressed, increase the tileSize by 8 and re-segment and process the image with the new tileSize before painting it to the canvas.
if (keyCode == DOWN) : If the down arrow key is pressed, reduce the tileSize by 8 and re-segment and process the image with the new tileSize, then draw it to the canvas.



Outcome

The video is in the file.